

Objective: Design and implement a multi-mode, interrupt-driven bicycle safety light. You will configure the microcontroller's internal hardware peripherals directly via memory-mapped registers.

Constraints

- All configurations and control must be performed using raw C pointers mapped to the address defined in the chip datasheet
- The use of software delay loops (e.g. for loops or `delay_ms()`) is strictly prohibited. Your CPU must remain free to process other events.
- Your code must modify only the specific bits assigned to your target LEDs and inputs. Accidentally overwriting neighboring register bits on shared GPIO ports will lead to a deduction in points.

Functional Modes: The system must cycle through three modes via a single physical button press:

1. **Mode 0 (Off):** System in standby; LEDs consume zero power.
2. **Mode 1 (Solid):** Constant, full-brightness illumination.
3. **Mode 2 (Blink):** A consistent 4 (or a random number) Hz blinking pattern.

Architecture Requirements

Part 1: Address Mapping and config (Memory Maps)

Locate the boundary addresses and offsets for your target GPIO ports in the datasheet. Write pointer definitions in C to configure the pins for input (button) and output (LEDs)

Part 2: Isolated State Transitions (Bit Manipulation)

Implement read-modify-write masking using bitwise operators to alter pin states. You must guarantee that writing to your LEDs does not corrupt or reset surrounding pin configurations on that port.

Part 3: Asynchronous Event Management (Interrupts)

- **The External Interrupt:** A hardware edge-trigger must instantly capture a press, update the system mode variable, and return.
- **The Peripheral Interrupt:** A hardware clock timer configured to fire periodically. The interrupt service routine must read the current mode variable and handle the timing patterns for the blinking mode.